

IKA: 基于语言机制的内核模块隔离和热升级方法

1. Abstract

传统操作系统内核因模块间共享地址空间与复杂依赖关系，导致内存错误（如缓冲区溢出、释放后使用）易引发系统性故障，同时也难以对模块实施热升级。已有的硬件隔离方案（如虚拟化扩展）因性能开销与灵活性不足难以适用于内核模块，而新兴的利用安全语言实施隔离和热升级的技术缺乏通用性。

本文提出了IKA，一个通用、灵活、低开销的隔离内核架构，通过Rust语言提供的类型与内存安全，实现语言驱动的内核模块隔离与热升级。IKA有几个关键的新颖之处，首先，基于系统分解和模块化思想，IKA将隔离所需的基础设施与隔离模块的实现解耦，这使得IKA可以轻松应用到不同的操作系统中，然后，IKA将对低级内核资源的所有不安全操作进行封装从而为隔离模块提供统一的安全抽象，进而实现灵活、低开销的内存隔离和故障隔离。最后，IKA利用状态数据的分离存储实现隔离模块的热升级，IKA还引入了优化的任务检测方法，进一步减少因升级而导致的服务暂停时间。

基于IKA，我们实现了宏内核AlienOS，我们还将该IKA应用到Linux系统中。实验表明：1) 基于隔离内核架构开发的隔离域可以检查和捕获常见的内存错误；2) Linux隔离驱动的I/O吞吐量平均仅下降1%-2%；3) Alien和Linux支持组件级别的热升级，服务暂停时间在100-200us。

2. Introduction

现代操作系统内核作为计算机系统的核心，承担着管理硬件资源、调度任务和提供基础服务的关键职责。传统单体内核设计通过共享地址空间实现模块间高效协作，却以牺牲安全边界为代价。以Linux为例，其庞大的代码库（截至2022年，5.11版本已包含约3000万行C代码【1】）中，约60%为设备驱动模块，而内存安全问题（如缓冲区溢出、释放后使用）占据了内核漏洞的绝大多数比例。研究统计表明，驱动模块贡献了绝大多数的内核安全缺陷【2】，这些缺陷往往通过复杂的模块依赖链蔓延，引发系统故障。例如，2021年Linux内核的TIPC协议栈堆溢出漏洞（CVE-2021-43267【3】）导致远程攻击者可触发系统崩溃，凸显了模块间隔离机制的迫切需求。

为应对这一挑战，学术界与工业界提出了多种隔离方案。硬件辅助方法（如Intel EPT内存虚拟化【4】、ARM TrustZone【5】）通过将内核划分到不同的虚拟机中或者特定的安全区域实现系统级别的隔离，但在内核模块之间的隔离上面临根本性限制：频繁的上下文切换导致性能开销显著，依赖特定硬件特性导致这些方法缺乏足够的灵活性。FlexOS【6】与CuriOS【7】等研究尝试通过新兴硬件（如intel MPK【8】）提供的高效、细粒度隔离接口实现unikernel中的组件隔离，但其架构与宏内核设计不兼容。软件层面的替代方案同样存在瓶颈——基于Java、C#等内存安全语言的系统（如Spin【9】、Verve【10】）依赖垃圾回收机制，其不可预测的gc时间与高速存储设备的微秒级延迟需求严重冲突；形式化验证方法（如seL4微内核【11】、CertiKOS宏内核【12】）虽能通过数学证明消除代码漏洞，但验证成本随代码规模呈指数增长【13】，难以扩展至复杂驱动模块。

近年来，Rust语言【14】因其独特的内存安全特性与零成本抽象能力，为系统编程领域注入了新的可能性。通过编译时所有权检查与借用规则，Rust无需垃圾回收即可杜绝数据竞争、悬垂指针等内存错误，这一特性与内核模块隔离的需求高度契合。最近的研究成果使用Rust开发新的系统，探索新的操作系统开发范式和基于语言机制的隔离和组件热升级方法【15】【16】【17】，然而，这些方法都不够通用和灵活，无法同时支持内核模块之间的内存隔离、故障隔离以及模块的热升级，也无法应用到其它操作系统中。

内存隔离是模块之间隔离的关键属性。其目标在于通过硬件或软件手段，防止一个执行单元的错误或恶意行为影响其他单元的内存数据，从而提升系统的安全性和隐私性。同时，内核模块仍然需要高性能的跨模块通信以保证内核的效率。因此，新的隔离方法必须支持低开销的内存隔离。redleaf【18】通过Rust语言定义的轻量级隔离域支持低开销内存隔离，然而，其开发的隔离域依赖由unsafe rust实现的可信库，一定程度上破坏了Rust语言提供的安全保障。其它的OS【redox】【RustyHermit】不支持内核模块之间的隔离。

故障隔离对提高内存隔离的有效性有很大的作用。当内核模块在运行时因为违反内存隔离属性或其他原因而出现错误时，模块有责任将错误限制在内部而不蔓延到其它模块中，并且模块需要把发生错误期间分配的资源释放，以避免可能发生的资源泄露和死锁问题。【safe driver】使用额外的运行时来记录模块对内核资源的分配操作以及其释放操作，从而实现资源的正确释放，这需要开发者的大量手工工作。redleaf【】则通过保存执行流的状态并将程序重置到未发生故障前，然后重新启动故障的模块来实现故障隔离，但这可能会丢失模块内部的状态数据。

对内核模块实施热升级是一项富有挑战的任务。支持热升级允许对运行良好的模块进行升级以增加额外的功能，或者对发生过故障的模块进行替换以修复已存在的错误。已有的热升级方法要不只能工作在较小的补丁上【】【】【】，要不就是针对特定的内核模块【】【】【】，缺乏一种有效的机制为不同的模块实现统一的热升级方法。Theseus【】采用了无状态模块设计，将模块状态转移至用户任务来实现模块的热升级，这一范式与商用内核中广泛存在的状态化模块（如文件系统缓存、网络连接跟踪）难以兼容。

为了同时支持内核模块之间的内存隔离、故障隔离以及模块的热升级并深度探索它们之间的关系。我们提出了IKA，一个通用、灵活、低开销的的隔离内核架构，旨在实现以下目标:1): 低开销: IKA支持模块的高效、灵活和细粒度的内存隔离; 2):高可靠性: IKA支持模块内的故障隔离; 3):组件级热升级: IKA支持对隔离模块进行组件级别的热升级; 4): 高通用性: IKA的隔离和热升级适用于不同的模块，可以应用在不同的操作系统系统中。

允许IKA应用在不同的操作系统中，包括使用Rust开发的操作系统和Linux内核中富有挑战性。IKA有几个关键的新颖之处，首先，基于系统分解和模块化思想，IKA将隔离所需的基础设施与隔离模块的实现解耦，这使得IKA可以轻松应用到不同的操作系统中，然后，IKA将对低级内核资源的所有不安全操作进行封装从而为隔离模块提供统一的安全抽象，进而实现灵活、低开销的内存隔离和故障隔离。具体而言，IKA将内核分解为一个小的可信计算基（TCB）与隔离域（Domain）两部分，TCB允许使用rust中unsafe部分，它被分解为runtime 和 domain lib，它们共同提供隔离和热升级所需的基础设施。隔离域则完全基于 safe Rust 实现。最后，IKA利用状态数据的分离存储实现隔离模块的热升级，针对无状态和有状态模块，IKA引入了优化的任务检测方法，进一步减少因升级而导致的服务暂停时间。

为验证IKA的通用性与实用性，我们使用Rust中实现了宏内核原型Alien，其中TCB仅占代码库的26%(第5.1节)，完全由safe rust编写的隔离域（覆盖任务调度、文件系统、网络协议栈等核心功能）占代码库的74%，我们将IKA应用到Linux系统中，将部分Rust实现的驱动改造为隔离域。实验表明：基于隔离内核架构开发的隔离域可以检查和捕获常见的内存错误；Linux隔离驱动的I/O吞吐量平均仅下降1%-2%；Alien和Linux支持组件级别的热升级，服务暂停时间在100-200us。

本文的主要贡献总结如下：

- 1. 提出了基于安全语言Rust的内核模块隔离架构IKA，通过语言级别的内存安全机制替代传统硬件隔离，实现了轻量化、细粒度、低开销的模块隔离
- 2. 结合模块隔离设计了基于状态数据分离存储的热升级方案，支持无状态域和有状态域的动态替换
- 3. 实现了包含多个隔离域的宏内核原型Alien，在Linux中应用IKA并完成常见驱动模块的隔离改造，验证了方案的通用性与实用性
- 4. 完全基于语言机制的隔离确保了隔离性能开销控制在1%-2%左右，低于硬件方案

3. Background and Motivation

	Type and memory safety	Memory Isolation	Fault Isolation	Live Update	Use in Linux
Verve(C#)/Biscuit(Go)	✓	✗	✗	✗	✗
Tockos/Asterinas	✓	✗	✗	✗	✗

	Type and memory safety	Memory Isolation	Fault Isolation	Live Update	Use in Linux
Theseus	✓	✗	✓	✓	✗
Redleaf	✓	✓	✓	✗	✗
IKA	✓	✓	✓	✓	✓

在本节中，我们简要介绍当前内核模块隔离和热升级的现状，并强调实施模块隔离和热升级面临的挑战。然后，我们总结了最近几个基于安全语言的系统设计以及它们在实现模块隔离和热升级方面的局限性。

3.1 Isolation and Live update In OS

在计算机发展早期，受限于性能有限的设备，在面对安全性和高性能的选择时，操作系统开发人员不得不选择后者。随着新的CPU架构【】【】和高速设备【NIC】【NVME】的出现，计算机性能取得了巨大进步，设备性能已经不再是限制内核开发的主要因素。几十年来，许多用户级和内核项目通过以下几种途径探索了实现内核模块隔离和热升级的可能性1): 基于硬件虚拟化或其它特定硬件机制【】【】【】【】2): 微内核化与用户空间技术【】【】【】【】【】，3): 基于编程语言的类型系统 and 安全性【】【】【】【】，4: 软件故障隔离【】【】【】。

从功能上看，隔离主要包含内存隔离和故障隔离，内存隔离是确保不同执行单元（如进程、内核模块或子系统）之间内存访问互不干扰的核心机制，故障隔离则将局部故障的影响范围严格限制在发生点周围，通常，故障隔离需要建立在内存隔离的基础上。它们是提高系统安全性和稳定性的关键。

Google【】、Microsoft【】和Apple【】的报告显示，内存安全问题占据了它们发现的漏洞的60%-70%。由于一贯的性能考虑，这些系统中大量使用不安全的C/C++进行编程。解决内存安全问题的直接手段是实施隔离，硬件级内存隔离（如虚拟内存和特权级划分）为用户态程序提供了基础保护，然而，用于地址空间隔离的进程抽象开销太大，为此，新的内存标记【MPK】和指针验证技术【arm MTE】【ARM PAC】允许用户在进程内部进行更细粒度隔离。糟糕的是，商品CPU架构仍然没有在安全性和稳定性要求更高的操作系统内核中引入高效、实用的隔离硬件。

<https://alexgaynor.net/2020/may/27/science-on-memory-unsafety-and-security/>
<https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html>
<https://source.android.com/docs/security/test/memory-safety?hl=zh-cn>
<https://www.chromium.org/Home/chromium-security/memory-safety/>
<https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>

虽然现代内核中开发了大量工具【ASLR】【Stack Canarie】【DEP】来检查和处理许多常见的内存安全漏洞，但这些工具并不是针对单个内核模块的。为了尽可能降低包含大量内存错误的非核心模块对系统的影响，人们利用虚拟化技术将这些驱动转移到其它CPU核心上单独运行【】【】，或者将驱动卸载到用户空间【】【】，一些方法则通过软件故障隔离（SFI）和语言安全技术，完全在软件中实施隔离，然而，这些办法均因为开销过大、无法通用而没有广泛应用。因此，新的隔离方法应该通用并且保持低开销。

热升级是一种在不重启机器的情况下对系统进行更新的技术，这对操作系统的可扩展性有很大帮助。操作系统大都已经支持了模块的动态加载和卸载，但对热升级比较完善的支持仍停留在函数级别【】【】，虽然这已经足够应付大多数场景，但却无法处理对模块进行大量修改的情形。组件级别的热升级的3个难点在于1): 维护组件的状态，在更新后，组件以更新前的状态继续服务；2): 尽可能降低服务暂停时间，这会显著影响那些延迟敏感型应用【】【】3): 适用于不同的模块。操作系统模块之间复杂的关系和内部复杂的状态导致已有的支持组件级热升级的方案往往不能兼顾这3点。比如，【】支持对调度器模块进行热升级并具有较短的服务暂停时间，但是无法直接在其它模块中使用。

对操作系统模块隔离和热升级的探索并没有停滞不前，随着编程语言的演进，诸如Rust等兼具高性能和内存安全的现代语言正在从更底层解决内存安全问题。从Google的报告可以看到，随着开发转向内存安全语言，Android系统内存安全漏洞的百分比在6年间从76%下降到24%【1】，这也是Rust语言得以进入Windows【2】和Linux内核【3】的重要原因。除此之外，高性能安全语言还为实现高效、轻量化且细粒度的隔离和组件级别的热升级提供了新思路，人们有望在基于编程语言的类型系统和安全性的途径上找到一个合适的方法。

3.2 Limitations of Existing Work

近年来，许多工作致力于探索基于安全语言的新的操作系统设计方法，包括微内核、宏内核、嵌入式系统和unikernel等内核形态。表1总结了它们与IKA的比较。

大多数安全语言依赖垃圾回收机制(比如C#【4】、Go【5】)，相比利用所有权【6】和线性类型系统【7】实现类型和内存安全的Rust语言，它们无需手动分配/释放内存，也不需要关注所有权和生命周期等复杂的编程概念。尽管优势明显，但基于垃圾回收的安全语言在实现低开销的隔离方面仍有困难。一些工作[Verve] [Biscuit]使用基于垃圾回收的语言重新实现操作系统，这些工作证明了现代高级语言可以从底层设计上降低操作系统内核发生的错误并在高层简化对操作系统的形式化验证工作，然而，由于这些语言通常不提供低级硬件交互的接口，这导致需要修改其运行时，而且，其垃圾回收器存在不可预测的暂停时间，这些缺陷造成了无法在商用操作系统中得到支持。

一些研究转而使用与C性能相近的安全语言Rust来开发新的系统，从而获得其高性能和安全属性。【Redox】

【RustyHermit】【Drone】【Bern】等用Rust实现了不同架构的内核，不幸的是，这些系统都只是简单地利用Rust作为C的替代品。【Tockos】【Asterinas】更进一步，它们将内核分为安全的世界和不安全的世界，其中较小的不安全的世界允许使用Rust中的不安全部分以处理面向硬件的低级操作并向安全世界提供安全的抽象，而安全世界则完全由安全的rust构成，从而获得Rust的安全保证。然而，这些系统没有探索Rust在类型和内存安全之外的功能，比如模块隔离和热升级。

【Redleaf】首先在基于Rust实现的微内核中引入了模块隔离，从而向前迈进了一大步。

其核心抽象是语言支持的隔离域(Domain)，这些域作为独立的隔离单元，可动态加载和安全卸载，并且崩溃时不会影响到其他域的执行。与【Tockos】【Asterinas】类似，它要求开发的域中不能包含unsafe代码，然而，其开发的大多数域都依赖由unsafe实现的可信库，这一定程度上降低了Rust语言对域提供的安全保证。Redleaf还依靠状态检查点机制和隔离域重启来实施故障隔离，这可能会造成隔离域内的状态丢失。

基于高级编程语言进行设计，【Theseus】重新思考了操作系统的结构和状态管理问题。通过Rust的类型和内存安全特性，它将许多传统由操作系统负责的任务(如内存管理和资源回收)转移到编译器层面。这种内在化设计

(Intralingual Design)减少了操作系统本身需要维护的状态量，简化了状态管理，从而可以以更简单的方式对内核模块进行升级和替换。然而，其热升级的可实施性依赖于无状态的模块设计，这与绝大多数包含大量状态的操作系统模块实现相悖。

至关重要的一点是，这些系统都缺乏通用性，即使它们在自己的系统上工作良好，却无法快速应用到新的系统中更不用说和已有的系统共存。

3.3 Design Goals

在这些工作的启示的基础上，IKA旨在提出一个通用的内核模块隔离和热升级方法，以探索安全语言Rust对操作系统安全性、可靠性和可扩展性的促进作用。我们将以下要求视为IKA的目标：

低开销 内核需要高性能，隔离开销是隔离方法是否被其它系统采纳的首要标准。在Redleaf【8】的启示下，IKA使用基于语言的方法实施模块间的内存隔离，从而避免传统硬件方法中的上下文切换或特权级切换开销，进而支持低开销的模块间内存隔离。为了更进一步减少Redleaf中隔离模块的通信开销，IKA需要避免引入额外的状态检查点机制。在此基础上，IKA就可以实现高效、灵活、细粒度的内存隔离。

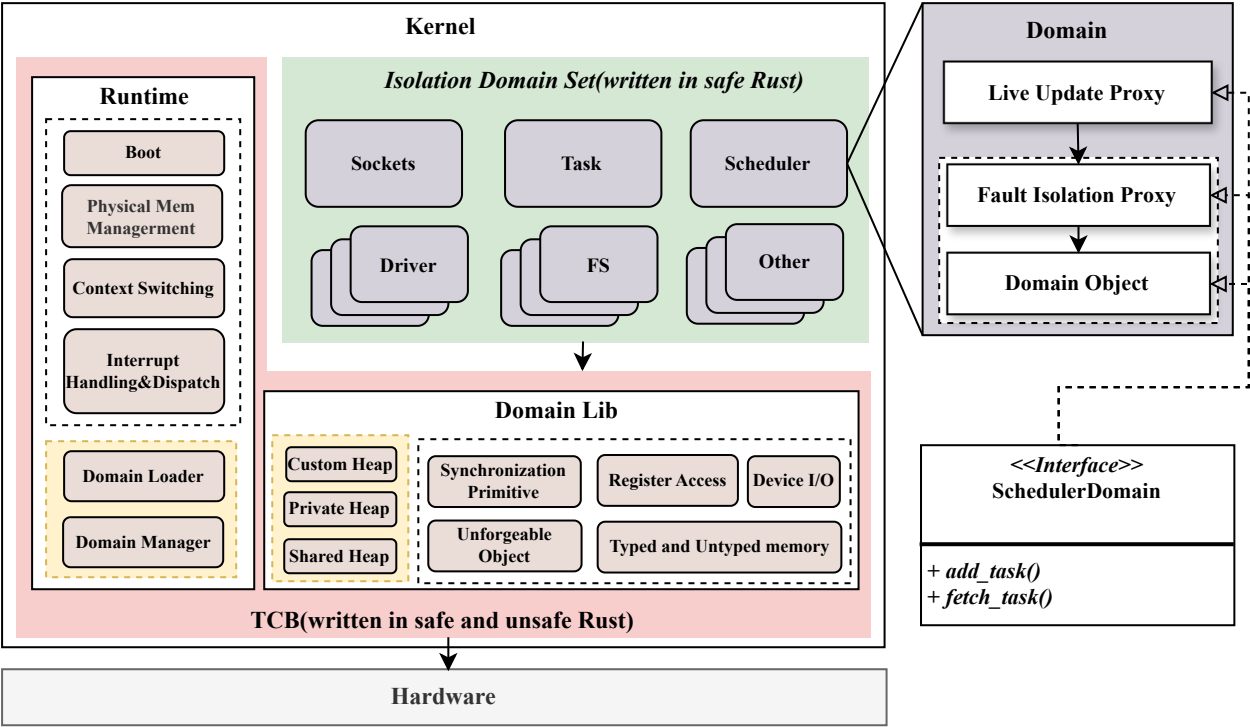
高可靠 故障隔离可以提高模块的可靠性。IKA应能捕获模块内部发生的故障，使其不会蔓延到其它模块中，并保证模块的正常运行。在故障恢复期间，IKA需要释放故障期间分配的内核资源，比如内存和锁，从而防止资源泄露和死锁。与【safe driver】中使用额外的运行时和【redleaf】中重启故障模块不同，IKA的故障隔离是一种无损语言级方法，这不需要重启故障模块从而导致模块内部状态丢失。

组件级热升级 IKA支持对隔离模块实施热升级。与【Theseus】不同，IKA不限制模块必须是无状态化的。此外，与【kpatch】只能进行函数级别升级或者【pluginsched】针对调度器进行升级不同，IKA不为特定的模块制定热升级策略而是提供统一的热升级方法，这意味着IKA的热升级方法应该具有足够的表达能力、并且适用于不同的模块。具体而言，IKA需要提供组件级别的热升级方案，并且保证模块内部的状态数据的一致性。此外，为了减少服务暂停时间，IKA要以高效的方式检测模块可升级的时机并快速完成模块替换和状态同步。

高通用性 IKA在设计上不应该依赖特定的系统功能，这意味者它不会被限制在特定的模块或特定的操作系统中。IKA需要分解实施隔离和热升级所需要的基础设施和具体的模块实现，从而允许其它操作系统在引入该方法时可以轻松地重用绝大部分工具。除此之外，作为一个可选项，IKA应该是非侵入式的，以避免对已有的系统进行较大的修改，更重要的，IKA应该可以和系统的已有部分共存从而允许系统逐渐过渡到新的设计上，例如，在Linux中，IKA中基于语言定义的隔离域应该和已有的LKM共存，开发者可根据对模块的安全性需求，自由决定使用新的隔离域抽象或者原有的LKM。IKA应尽力而为提供工具简化隔离域的开发、编译和使用，这有利于开发者将更多模块实现为隔离域。

4. Design

4.1 Overview



基于设计目标，我们设计了隔离内核架构——IKA。图()展示了IKA的整体架构。整个体系结构将内核分为两个部分：允许使用unsafe rust的TCB，包括Runtime和 Domain lib和完全使用safe rust的隔离域集合。

在IKA中，内核的绝大部分功能由许多不同的内核模块(称为隔离域)实现，这些模块可以在运行时进行加载、卸载和热升级。隔离域是软件定义的隔离单元，基于Rust语言提供的类型和内存安全性实现隔离，而不是硬件保护。每个域通过其域接口向外提供服务，在内部，隔离域依赖domain lib中的安全抽象，域的所有的代码完全由safe rust编写。每个域拥有一个私有堆用于内部堆分配、全局共享堆引用用于域间通信(第4.3.1节)，此外，还有一个自定义堆用于保存

域的状态数据(第4.4.1节)。隔离域基于函数调用通过域接口进行域间通信。

Runtime和Domain lib共同构建了实施隔离和热升级所需的基础设施。在内部，它们都被分解为与系统相关部分和不相关部分。Domain lib作为一系列小组件被隔离域所使用，它的功能包含两部分：

- 1. 将所有需要使用unsafe rust实现的不安全操作进行安全抽象，向隔离域提供对内存或硬件资源的安全访问接口（系统相关）；
- 2. 提供隔离和热升级所需的私有堆、自定义堆和共享堆（系统无关）；

Runtime是作为一个可运行单元存在的，这与Domain lib不同。其功能同样包含两部分：

- 1. 对系统进行初始化，管理物理内存，执行特权级的切换并处理硬件中断和异常。(系统相关)
- 2. 负责加载、卸载和升级隔离域，并在系统运行中管理隔离域。(系统无关)

APIs	Descriptions
CoreFunction:: *	调用Runtime中的功能
HeapAllocator::alloc/dealloc	分配私有堆数据
SharedHeapAlloc::alloc/dealloc	分配共享堆数据
storage::insert/get/remove	存储状态数据
Mutex/SafeIORegion/SafePtr/BasicFrame/...	内核资源的安全抽象

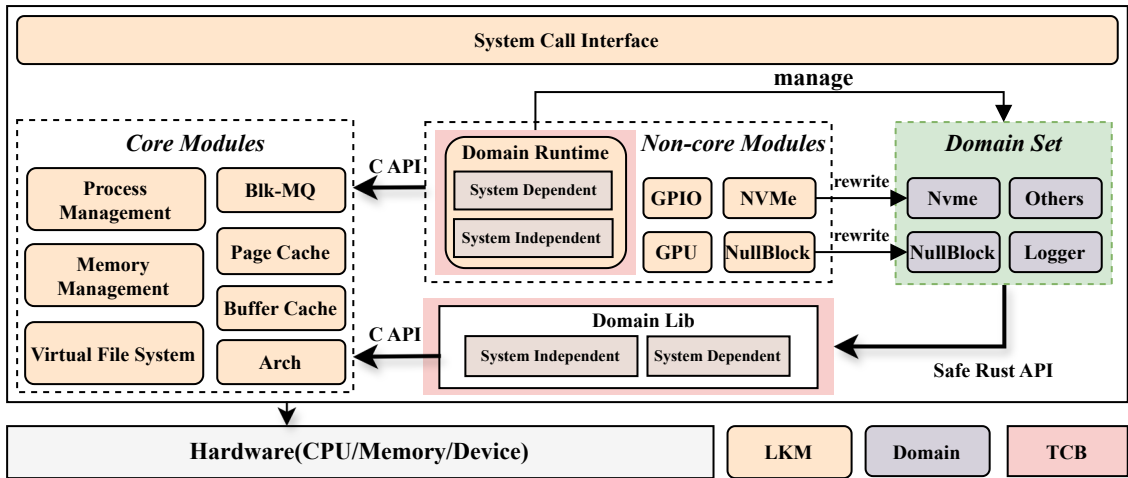
APIs	Descriptions
DomainResource::insert_page_map/free_page_map	管理域分配的物理内存
storage::create_domain_storage	为域创建状态存储堆
storage::remove_domain_storage	移除域的状态存储堆
storage::move_domain_storage	更新域的状态存储堆
DomainLoader::load/unload	加载域到内核地址空间/卸载域
DomainManager::register_domain/query_domain	注册域/查询域

4.1.1 Trust Model

IKA的隔离方法建立在Rust语言提供的安全保证上，Rust语言的safe代码提供了四个对于在软件中提供内存隔离至关重要的保证【】：1)它们不允许指针算术，并要求代码获取的任何引用要么是由于分配或函数调用而生成的；2)它们检查数组访问的边界，从而防止由于缓冲区溢出（和下溢）而导致的杂散内存访问；3)它们不允许访问空对象，从而防止应用程序使用未定义的行为来访问应隔离的内存；4)它们确保所有类型转换都是安全的（并且在兼容对象之间）

这些安全保证是IKA实现模块隔离的基础，此外，IKA的隔离还依赖1) 用于实现Rust语言的类型和内存安全的编译器；2) Rust的core和alloc库，这些库中使用unsafe rust实现了许多基础类型。3) 隔离基础设施的正确实现。IKA不处理硬件导致的内存安全问题，比如内存管理单元MMU，中断控制器等，我们假设这些系统设备不是恶意的。

4.2 Decomposition and abstraction



4.2.1 Decomposition

IKA应该是通用的，这意味着其可以广泛地应用在其他系统中，并且可以简单地修改。正是因为缺乏对内核不同部分的分解，已有的隔离方法无法快速地应用在新的系统中，甚至无法快速实施对其它内核模块的隔离。为了做到这一点，IKA提倡将整个内核分解为两个部分：允许使用unsafe rust的TCB和只能使用safe rust的隔离域集合。

域是被独立编译并被动态加载的，这有点类似与LKM，但又与LKM有明显的差异。IKA使用Rust的trait系统为每一类域定义一个trait(一组函数的集合)从而提供一个接口的抽象，这基于对系统的观察：内核中的模块通常以一组函数请求其它模块服务，并导出一组函数给其它模块服务。每个域需要实现特定的trait。在实现层面上，域被强制从unsafe代码中解放出来，因为Rust的类型安全和内存安全无法在unsafe代码中得到有效验证，因此域的依赖项只能包含Rust的core和alloc安全集合以及domain lib的安全抽象。换句话说，IKA的隔离域具有在实现时定义明确的边界，并在运行中持续存在。域对外只暴露明确的接口，内部实现细节对外隐藏，域之间的交互通过接口调用实现。

IKA将TCB进一步分解为runtime和domain lib。domain lib中的安全抽象是建立在对域的分析基础上的：在使用Rust进行操作系统开发时，针对低级内核资源的一些操作不得不使用unsafe rust进行实现，但是域不允许使用unsafe rust，同时Rust语言中的内置类型也不足以表达这些资源。为此，在domain lib中，我们将编译器检查的不变量(无缓冲区溢出，无释放后使用)扩展到对低级内核资源的操作中，让这些资源被无缝地纳入到Rust提供的强大静态类型安全中，这是通过对低级内核资源和相关联的操作进行安全抽象(第4.2.2节)来实现的。对低级内核资源的安全抽象通常是与系统相关的，但是用于实施内存隔离的私有堆和共享堆(第4.3.1)以及用于实施热升级的自定义堆(第4.4.1)等基础设施是与系统无关的，因此，我们进而将domain lib分解为较小的组件，以允许IKA应用到新的系统中时重用这些基础设施。

通常，不同系统的初始化代码是不一样的，与domain lib一样，IKA也将runtime分解为较小的组件，从而将域的加载器和管理器等基础设施与其它部分分开来。为了让TCB尽可能保持小巧，以此让绝大部分内核功能受益于语言的类型和内存安全，我们建议减少runtime负责的功能，比如，runtime响应硬件中断，但是处理中断的功能在对应驱动域中实现，runtime进行任务切换，但是任务的生成和调度在不同的域中实现。不过，为了和现有的系统的兼容，IKA并不强制当前的系统具有较小TCB。我们可以将IKA作为现有系统的扩展(第5.1节)，此时只有采用隔离域形态实现的模块才能保证隔离和热升级。

分解带来了两个好处，首先，分解使得大多数内核模块可以被纳入隔离的范围。其次，实施隔离所需的基础设施与隔离模块的实现分离，这些基础设施就可以在不同的系统中被轻松重用。

4.2.2 safety abstraction in domain lib

仅使用安全的Rust代码无法将所有的内核模块实现为安全的域，比如，一些模块不可避免地需要unsafe代码处理内存读写或设备I/O。在domain lib的安全抽象部分，IKA将这些针对低级内核资源的不安全操作通过Rust语言进行封装和安全抽象，然后为隔离域暴露出安全的接口。具体而言，domain lib中对内存读写、设备I/O、指针操作、寄存器访问、同步原语等低级内核资源进行了安全抽象。尽管具体的实现会因为系统的差异而有所不同，但IKA提供了一些通用的指导原则和类型抽象。

最小化和集中原则：domain lib只会对最底层的低级内核资源和相关联的不安全操作进行安全抽象，而可以由这些资源和安全接口组成的高级抽象和接口则是由域自行实施，因为这些不再需要unsafe rust。与redleaf【】中每个隔离域依赖不同的可信库相反，IKA通过domain lib中的安全抽象为所有域提供了访问低级内核资源的统一途径，避免了为大量外围设备驱动编写可信库的需要。而且，IKA尽可能将更高级的抽象留到域中实现，这在很大程度上减少了domain lib所包含的代码，从而降低了对其内部unsafe代码的审查难度。

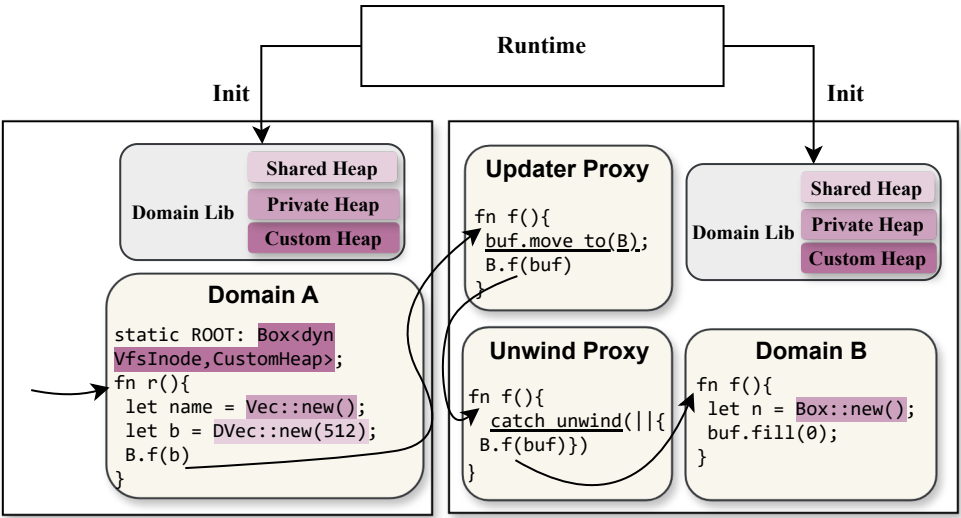
内核资源的安全回收：所有在分配后需要释放的资源都要实现Rust内置的 `Drop` trait，一方面，这使得内核资源的管理从开发者的手工工作变成了编译器的自动实施。另一方面，在域发生故障时，故障处理流程可以正确回收这些资源。

域状态数据的安全存储 域使用domain lib中的安全抽象实现其功能，这意味者一些内核资源可能构成域的状态数据。对于绝大多数资源来说，它们是平凡的数据结构，因此不会进行堆分配。然而，少部分安全抽象内部可能会分配堆空间，默认情况下，这些堆分配发生在域的私有堆上。在域将资源作为状态数据来看待的情况下，这些资源就需要被分配到自定义堆当中，从而可以在域热升级后继续存在(第4.4.1)。为此，domain lib为这些类型添加了一个额外的构造函数，在这个额外的构造函数中堆分配被指定到域的自定义堆上。

内存访问的安全抽象 domain lib将域可以访问的内存分为两类，类型化内存和无类型内存，两者可以互相转化。Domain lib定义了 `FrameTracker` 来表示无类型的内存，其对应到一段连续的物理地址区间。在系统中，无类型内存主要用于四个场景中，1):用于域的细粒度堆分配。2): 用于与硬件交互，读取硬件数据或写入数据到硬件中。3): 用于映射任务地址空间和访问用户数据。类型化的内存包括域的代码、数据段、设备地址空间，Rust堆等。在域被加载后，无类型内存自动转变为类型化内存，并由Rust的类型系统接管。例外之一是设备内存，无法从无类型内存转变到Rust中的对应类型，domain lib引入一个 `SafeIORegion` 的类型来表示设备内存，与其它类型的内存一样遵守Rust的所有权规则。在运行过程中，域可以将 `FrameTracker` 动态地转换为具体的数据类型，但是必须满足两个条件：1) 无类型内存的地址需要符合数据类型的内存对齐规则；2):无类型内存大小不能小于数据类型的大小。

指针对象的安全抽象 在初始域集合被加载到内核后，一些驱动域从runtime获得设备信息，并接管硬件设备。通常，设备资源位于特定的内存地址区间，因此对资源进行封装的数据结构需要一个指向其地址的指针，在safe rust中，域无法操作裸指针，但却可以创建一个裸指针。为了防止域传递一个无效的裸指针，domain lib引入了不可伪造对象，它将裸指针作为一个64位大小的 `u64` 基本类型看待，但被包装在 `SafePtr` 类型下，这有点类似于类型别名，但在Rust中 `SafePtr` 是一个与 `u64` 完全不同的类型，`SafePtr` 的构造函数和大部分指针访问函数是unsafe的，这意味着域无法在safe的代码中随意构造和使用这个对象，但是runtime可以构造，domain lib可以访问。在Linux系统中，不可伪造对象还被用于处理回调函数的重定向(第5.2.2)中，因为这些回调函数的参数大多由裸指针表示。

4.3 Domain Isolation



4.3.1 Memory Isolation and communication

IKA通过将域的实现限定在Rust的安全子集之内并提供domain lib以封装域所需的unsafe 操作来确保域获得了Rust语言提供的安全保证。然而，这还不足以实现完全的内存隔离，因为域仍然可能通过域接口传递包含复杂层次和引用关系的对象。为了解决这个问题，和之前的许多工作类似【】 【】 【】， IKA引入私有堆和共享堆来满足域自身所需的堆分配和域间通信所需的数据传递。

私有堆 域的内部需要进行堆内存分配，IKA通过为每个域建立一个私有堆来隔离域对堆内存的访问。与C语言的堆分配不同， Rust的堆分配由一个全局的堆分配器进行管理，而每个被独立编译的域都需要声明其堆分配器，IKA将这个分配器视作域的私有堆。runtime管理域的功能之一是为域分配较大的页，并记录页的域所属权。在域内部，这些页被全局堆分配器用来分配较小的对象。采用两级分配器有助于runtime在系统运行期间快速为域分配内存资源，并且在在对域进行卸载或更新期间可以安全且快速地回收这些资源。由于域只能从自己的私有堆上分配对象，域不能存在指向其它域的私有堆的指针。

域间通信和共享堆 通常，单个域可能并不能完成所有功能，需要通过域接口访问其它域的功能。为了阻止域接口成为私有堆对象的逃生通道，IKA为所有域提供一个全局共享堆，并规定域间通信的数据共享通过共享堆完成。然而，单靠共享堆并不能约束域间传递数据的方式，比如，域可能将私有堆数据放置在共享堆中进行传递。为此，IKA引入了特殊的共享堆基本数据类型 `DBox<T>`，并在此基本类型上构建其它数据类型。`DBox<T>` 通过Rust的auto trait特性【】来实现1): 共享堆上分配的对象不能包含指向私有堆的指针; 2): 共享堆上分配的对象不能存储私有堆对象。此外，IKA不允许域接口使用可变对象引用，虽然可变对象方便对数据修改，但会使得实施域的故障隔离变得困难。相反，IKA使用不可变引用共享数据或通过移动共享堆对象所有权来实现对数据的修改。

`DBox<T>` 与rust的内置类型 `Box<T>` 类似，但引入了一个额外的字段来记录其所属的域。在进行域间通信时，如果需要移动所有权，该字段会被更新为新的域标识符并在调用完成后恢复。移动所有权意味着调用者失去访问权。在域被卸载或更新时，runtime扫描被分配的共享堆对象，回收所有权属于该域的共享堆对象。

直观来看，域间通信就是对域接口函数调用，这比那些需要进行特权级切换的方式【】 【】要更高效。从我们实现的原型系统和在linux上的应用来看，对域接口施加的安全限制，并不会对域的功能开发产生额外的负担。

4.3.2 Fault Isolation

鉴于操作系统的复杂性，消除所有错误是不现实的，但内核需要限制错误的影响范围，防止发生故障的模块破坏其它模块。在IKA中，内核分解和域的内存隔离属性促进了故障隔离的实施。分解意味着故障发生在较小的单个隔离域中，内存隔离属性确保了发生故障的域不会擅自修改其它域的内存。为了实现域的故障隔离，IKA需要解决的一个核心问题是: 如何正确处理故障?

Rust语言使用panic来表达程序故障，根据编译时配置，程序可以选择在发生故障时直接终止程序的执行(abort)或者对执行流进行堆栈展开(unwind)。为了捕获并处理域的故障，IKA依靠Rust语言内置的panic钩子和故障捕获点。在编译时，IKA会修改域的编译选项以启用unwind策略，runtime在域加载时将编译器产生的DWARF【】信息一同加载。

在域内部，IKA为每个域定义panic钩子，当域中出现故障时，执行流将会跳转到panic钩子中进入故障恢复流程。在钩子中，IKA会沿着栈帧向上展开，即从当前的栈帧逐步退回到调用方的栈帧。在每一个栈帧中，它会依次调用栈中每个活跃对象的 Drop 方法，确保资源被释放，最后，unwind将会在故障捕获点处停下，结束故障处理。为了将故障隔离在单个域中，故障捕获点被定义在域的入口处，也就是在调用域接口的位置。

通过为内核资源实现 Drop trait，IKA可以在控制流的堆栈展开过程中自动释放这些资源，因此在处理完故障后不会泄露内核的资源，不会导致死锁。这种异常处理在C语言要困难的多，与【safedriver, redleaf】不同，IKA的unwind策略是一种无损语言级方法，编译器会确定何时可以安全地触发资源清理例程。

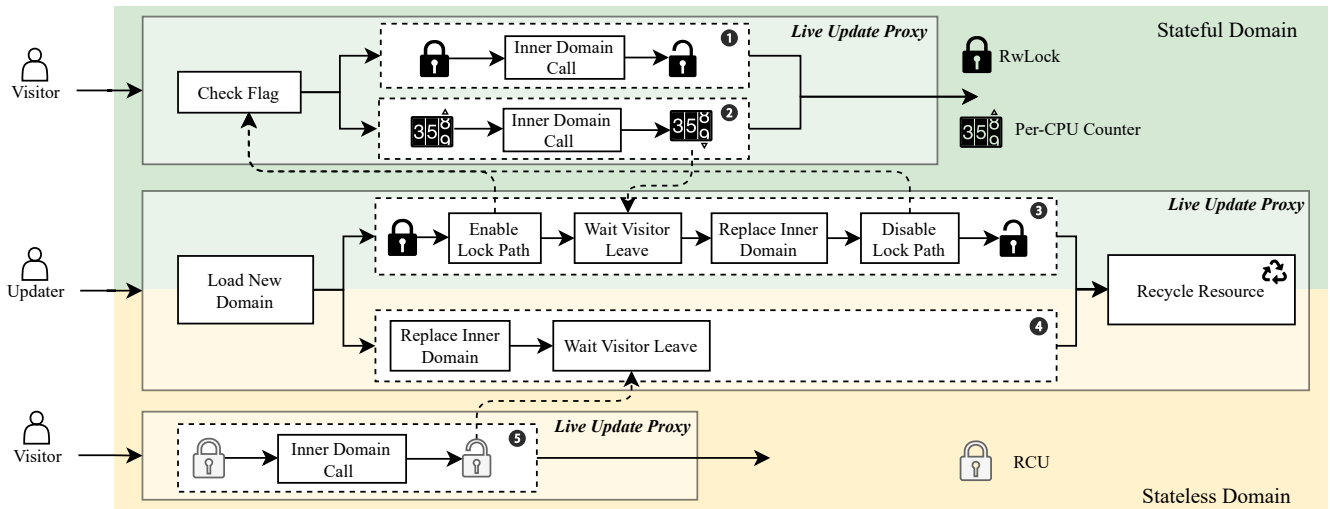
执行流的堆栈展开不会影响域的服务，其它位于域中的任务的执行流仍然正常进行。但由于域出现了故障，有很大概率是域的实现出现了错误，为了表达这一特征，在panic处理流程中，IKA发出警告信息并记录域发生崩溃的次数。在用户态，IKA暴露了一个 domainfs，向系统维护者展示系统中所有域信息，同时也包括域的故障计数。维护者可以根据故障计数判断错误的严重程度并决定是否对域进行更新修复。

不过，基于unwind的故障处理不是万能的，首先，rust的panic不能发现所有的故障，比如，堆栈溢出将会被MMU发现并被操作系统结束任务，硬件错误也不会被panic所发现。另一方面，当故障发生在域修改了一些内部状态之后，故障恢复将不能撤销这些修改，这可能会导致域出现更多的故障，此外，故障恢复期间如果系统内存不足将会导致故障恢复失败。这些问题已经超出了故障隔离的能力范围，因此，IKA将责任推送给系统维护者。

Fault Isolation Proxy IKA依靠故障隔离代理来自动实施故障隔离从而向外部隐藏处理细节。具体而言，故障隔离代理将真正的域对象进行封装，并且实现和域对象相同的trait，这相当于对域接口调用进行了一次中转。在内部，故障隔离代理将域对象的所有接口调用包装在一个特殊的函数中，这个函数使用Rust内置的特殊方法使得编译器生成故障捕获点。需要注意的是，相比【】在接口调用时保存任务寄存器信息，这种方法不会给域间通信带来较大的开销。故障恢复流程最终将暂停在捕获点处并返回一个错误以表征域内发生了故障，这样，发生故障的域就不会向上返回错误的数据。

ICA进一步扩展了故障隔离代理的功能，如果内部对象的调用返回了 Err(DomainCrash)，除了直接向上返回错误外，它还允许对特定的调用进行重新调用从而向调用者隐藏域故障情况。但重新调用并不是免费的，在故障恢复进行堆栈展开期间，由域接口传递到域中的共享堆数据会被释放，所以故障隔离代理无法使用相同的共享堆数据进行重新调用。一种可能的方法是在第一次发出调用时拷贝一份共享堆数据，但这种做法开销过大。因此，故障隔离代理只允许在域接口使用不可变引用共享数据的情况下进行重新调用，Rust编译器确保了不可变引用的数据不会被修改，而引用不会被释放。

4.4 Living Update



在像Linux这样的内核中做动态升级是困难的，最重要的原因是内核状态数据的复杂性，内核模块之间互相交织导致一个模块的状态可能被多个模块共享和修改。基于语言机制的模块隔离促进了域的状态数据隔离，域之间通过接口访问功能，因此域不会共享特殊的全局状态，每个域是独立的部分，它们的状态数据也是独立的。已有的工作【】大多将隔离和热升级视作两个独立的问题进行解决，或者像Theseus【】另辟蹊径以无状态模块设计解决热升级的问题。与这些工作不同，IKA将热升级和隔离结合起来一起考虑，这不仅可以大大简化热升级的实施，还不需要推翻现有的常见的有状态模块实现方法。

4.4.1 State saving and restoration

热升级的核心在于确保域在升级前后拥有相同的正确的状态。当深入地考察域更新前后所发生的变化，我们发现，对一些域的更新实际上不需要状态数据的保存和恢复，IKA把这种域称为**无状态域**。这里的无状态指的是域内部的数据是无关紧要的，不需要在热升级的时候考虑。对于其它的域，IKA称为**有状态域**，有状态包含两层含义：

1. 这些域内部确实保存了需要在更新前后保持一致的状态数据，比如任务管理、虚拟文件系统等
2. 这些域更新后可以从一个初始状态重新开始运行，但是这些域可能会独占机器的硬件设备，比如底层设备驱动程序

通过对有状态域的分析发现，域中分配的大多数数据都会很快被释放掉，但是对于域运行至关重要的状态数据则通常不会马上释放，并且很多时候都被保存在堆内存或者全局变量中，这些状态数据仅仅占据域数据的一小部分。

Plugsched【】正是利用了这一点，结合模块中特定API，它将模块中的状态数据以类似 pop 的操作移出旧的模块，然后以类似 push 的操作移入新的模块。然而，这种方法过于和模块API绑定，通用性比较差。IKA从另一个角度出发：

1. 既然模块的状态数据和普通数据不同，那么就直接在定义时区分两者的存储方式
2. 既然模块状态数据需要在模块的整个生命周期中存在，那么只需要将状态数据保存在一个可以持续存在的地方即可。

自定义堆 IKA利用Rust的自定义堆分配器实现状态数据的分离存储。自定义堆分配器允许将分配的对象存储在一块特定的堆内存中。为此，IKA为每个域引入了一个自定义堆分配器。domain lib向域提供了分配、删除和索引接口，分配接口用于域进行对象分配，索引接口用于域在升级后索引已经保存的状态数据。在升级期间，域的资源被运行时回收，包括其使用的私有堆、代码段、数据段等，而在自定义堆中保存的状态数据则被保留下来，自定义堆的域所属权会被更新为新的域，更新之后的域将直接继承这些数据。

自定义堆的引入带来了对象分配方式上的改变，IKA尽量保持接口简洁易用以降低开发负担。为了让隔离域支持热升级，域开发者需要1): 区分域内的状态数据和非状态数据。2): 使用domain lib提供的分配、删除和索引接口进行状态数据的管理。这种方式在通用性和灵活性上取得了折衷，开发者需要对代码做一些修改，但对所有的域都适用。不过，受限于是这是一种语言提供的机制，还暂时不能将其应用到已有的使用C实现的模块中。

4.4.2 State synchronization

Stateless Domains 针对有状态和无状态域，IKA应用不同的状态同步机制。在升级内核模块时，较大的服务暂停时间会影响系统提供的服务。最理想的情况是在更新时保持服务而不中断，而对无状态域的划分可以实现这一目标。域是无状态的，意味着域对外提供服务并不依赖自身存储的数据，这意味着在进行无状态域升级时进行以下操作：

1. 旧域仍然向外提供服务，并不会在更新时立即从系统中移除
2. 新域被加载到内核中，新的任务使用新域的服务
3. 在旧域中活动的任务全部离开后，runtime再卸载旧域，此后所有的服务由新域提供

RCU RCU (Read-Copy-Update) 【】是一种同步机制。它允许读者在不加锁的情况下读取共享数据，同时确保写者能够安全地更新数据，而不会影响正在进行的读操作。IKA使用Linux内核中流行的RCU机制来完成无状态域的升级。简单来说，IKA把无状态域视为需要并发访问的数据，域的普通访问者作为读者(图中路径 5)，域的更新者作为写者(图中路径 4)。这样就可以把RCU的优势完美应用到对无状态域的热升级中。当新的域加载到内核中时，旧的读者可以继续使用旧域提供的功能，而新的读者不用等待所有旧读者离开就可以直接使用新域提供的功能。域的更新者会使用RCU同步原语确保所有读者离开旧域，runtime就可以安全删除旧域所有的资源。

Stateful Domains 有状态域与无状态完全不同，其提供的服务依赖内部的状态数据，如果在更新期间有其它任务位于旧域中，并在更新域的同时修改了域中的状态数据，就会造成状态数据不一致，引发严重的错误。因此，有状态域的更新需要确保所有的任务已经离开旧域并阻止新的任务进入旧域中才能安全地重新初始化新域。

为了检查所有任务是否位于模块中并进行升级，大多数补丁工具(比如kpatch)使用Linux的Stop machine接口来让CPU进入静止状态【】，在状态静止期间，特定的CPU执行用户定义的处理程序，其余的CPU停止工作。Plugsched【】进一步使用任务并行和二分搜索来加速任务检查。但是Stop machine仍然造成了比较长的服务暂停时间，它还会让正在执行其它任务的所有CPU停止，即使这些任务跟需要升级的模块并不相关。造成这个问题的原因是现有的模块缺乏一个明确的边界，无法直接统计当前位于模块中的任务。

Dynamically enable lock 隔离的一个关键属性允许我们执行快速的任务检查然后进行升级：域接口构成了一个明确的边界。IKA引入了读写锁来进行同步，从而确保在更新期间没有任何任务位于旧域中。具体来说，不管是普通的访问者还是更新者，都会先获取锁再调用域接口。然而，当大量任务调用域接口进入域中，在每一个接口上都用锁带来的性能开销是巨大的。为了解决这个问题，IKA采用一种折衷的方法，具体而言，IKA为域增加了一个全局标记，在有状态域的正常运行期间，所有的域调用将会走一条没有锁的代码路径(图中路径 2)，此路径会对任务的进入和离开进行计数。当有更新者进行域更新时，更新任务将提前获取锁，并使能全局标记(图中路径 3)，后续的访问者因为全局标记被使能而走有锁的路径(图中路径 1)，但因为更新者已经获得锁，它们需要等待升级结束才能获得锁。

在更新者使能标记后，它统计所有CPU上的任务计数器从而确保所有访问者已经全部离开。如果计数器之和为0，意味着进入安全状态，可以对域进行更新。在这里，动态使能锁在多核情况下引入了一个微妙的竞争条件：如代码1所示，cpu2的更新者使能标记前一刻，cpu1的访问者1已经读取了标记，其会走无锁路径，但是在更新者统计任务计数前，访问者1的执行被中断打断，由于访问者1没有更新任务计数器，更新者会误认为可以进入安全状态，中断结束后访问者1才更新计数器并且进入到了旧域中。其它在更新者使能标记后读取标记的访问者不会受此影响，因为即使中断发生在检查标记和获取锁之间，中断结束后，它们依然会被锁阻挡。

IKA使用一个巧妙的方法解决这个问题，如代码2所示，首先，在访问者检查标记前，关闭当前CPU的中断，如果标记指示走无锁路径，那就在完成计数器增加操作后，重新开启中断，这样这段代码就不会被打断，如果有锁路径，那么在检查标记后就可以重新开启中断。然后，对于更新者，在统计任务计数前，它需要向其它CPU核心发送一次核间中断，并等待其它CPU响应。当其它所有CPU都响应了这个核间中断，意味着其它CPU上走无锁路径的访问者已经全

部完成了计数器更新，此时更新者才能正确地统计任务计数。runtime会调用新的域对象的初始化接口完成新域的初始化，然后将新的域对象替换掉旧的域对象从而完成升级。由于域的状态数据存储在自定义堆中，只需要将该旧域的自定义堆的标识符更新为新域即可完成状态数据的迁移。

```
1 visitor1(core1)    Updater(core2)    visitor2(core3)
2 check flag        lock()
3                  enable flag
4 -interrupt-              check flag
5                  calculate counter    lock()
6 inc counter
```

```
1 visitor1(core1)    Updater(core2)    visitor2(core3)
2 disable interrupt
3 check flag        lock()
4                  enable flag        disable interrupt
5                  check flag
6                  send IPI        enable interrupt
7                  calculate counter    lock()
8 inc counter
9 enable interrupt
```

Living Update Proxy 与故障隔离代理类似，热升级代理的目标是向外部隐藏具体的热升级处理逻辑。热升级代理会将故障代理进行封装，并且也会实现和域对象相同的trait。对于域开发者来说，它们使用其它域的方式是接口对象，因此，热升级代理和故障代理对域开发者来说是透明的。域间通信在热升级代理中进行第一次中转。热升级代理的主要功能由两部分组成，首先，针对无状态域和有状态域，热升级代理将内部对象封装到不同的同步数据结构中，对于无状态域，热升级代理使用RCU数据结构，对于有状态域，热升级代理使用per-cpu计数器和读写锁。其次，热升级代理在进行热升级期间进行对象的替换和状态数据同步。

5. Implemation

我们为IKA实现了用于支持模块隔离和热升级的基础设施，它包含3.5k行Rust代码。目前，这些基础设施作为一系列小组件存在，在不同的系统中，它们将分别被IKA中定义的runtime和domain lib所使用。除了第4节描述这些关键基础设施外，我们还实现一些额外的功能和工具来减少开发人员的工作。

Code Generator 我们观察到，对于所有的域来说，他们拥有相同的domain lib初始化代码，域的更新代理对象和故障代理对象与域对象实现相同的域接口，同类型的域会实现同一个域接口。解决代码重复实现的简单方法是引入一个IDL语言和编译器，编译器根据接口定义生成代码，这也是大多数类似工作的选择【】【】，但这会导致额外的学习成本，并且可能无法高效地和系统进行同步。我们摈弃了这种做法，相反，我们完全使用Rust来生成隔离域的模板代码，并使用Rust的宏来完成域代理对象的接口实现。如代码所示，在定义域接口时，开发者可以给接口添加一个Proxy属性，并指定域代理名称、更新代理对象使用的同步方法，以及域初始化参数类型。根据这些参数，我们定义两类域代理对象的具体实现，并导出两个宏接口。使用这两个宏接口，Rust编译器自动生成预定义代码。


```

1  #[proxy(BlkDomainProxy,RwLock,Range<usize>)]
2  pub trait BlkDeviceDomain: DeviceBase + Basic + DowncastSync {
3      fn init(&self, device_info: &Range<usize>) -> Result<()>;
4      fn read_block(&self, block: u32, data: DVec<u8>) -> Result<DVec<u8>>;
5      fn write_block(&self, block: u32, data: &DVec<u8>) -> AlienResult<usize>;
6  }
7  // export macro
8  gen_for_BlkDeviceDomain!();
9  define_unwind_for_BlkDeviceDomain!(BlkDomain);

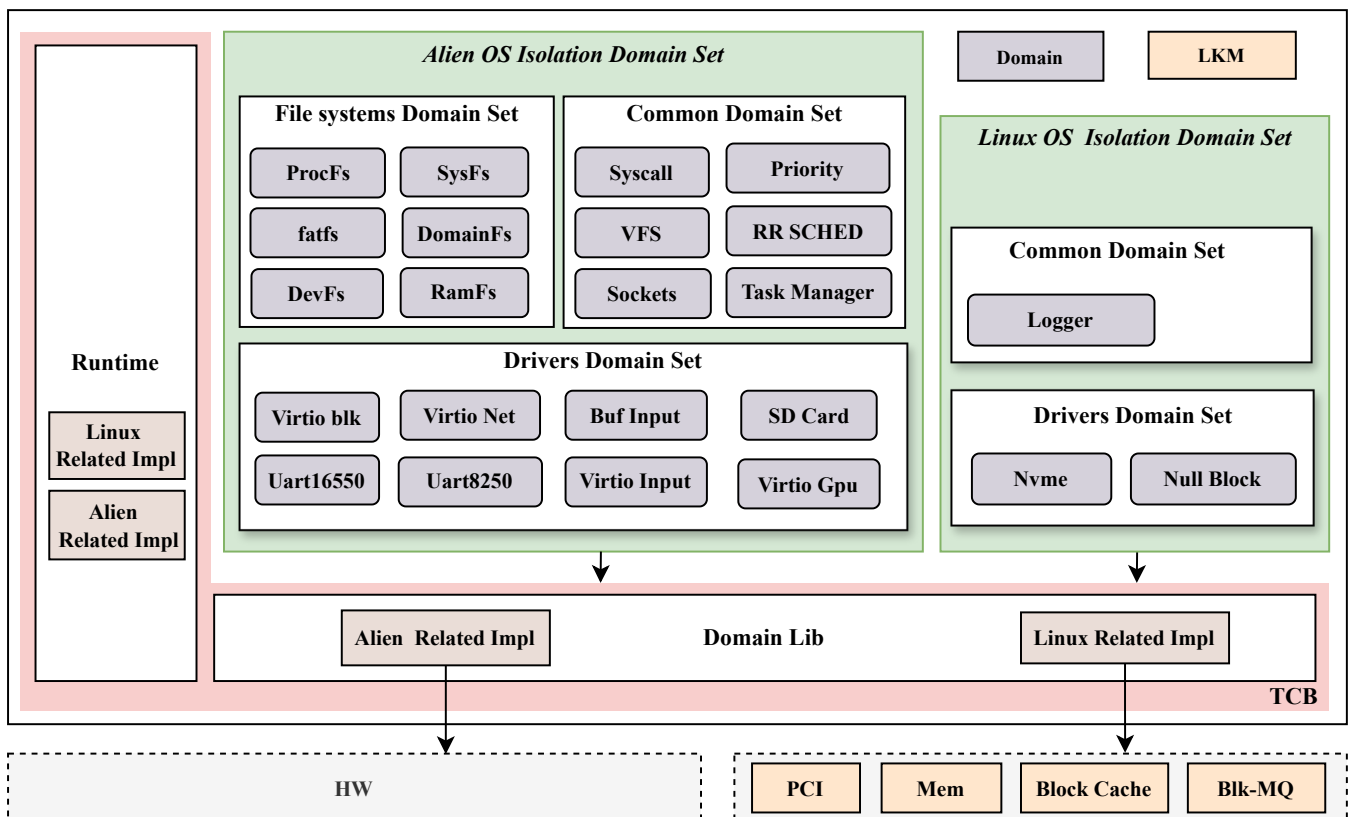
```

User Tool 为了简化开发者创建、实现和编译域的流程。IKA提供了自动化工具，开发者以交互式的方式选择不同类型的域，并提供域要实现的域接口，工具将会为用户创建一个Rust项目，包含一个库程序和一个可执行程序以及它们的基本依赖。开发者像开发库一样提供域的实现，不同的是，库不向外暴露接口，开发者在预定义的虚拟 `main()` 函数中返回实现了域接口的域对象。可执行程序并不需要开发者编辑，它会自动引用库程序。稍后，用户可以选择编译指定的域，自动化工具将会自动搜索域位置并编译其为位置无关可执行程序。

5.1 Case Study

为了验证IKA的可实施性，我们首先在riscv64平台上从零实现了一个原型系统Alien OS，除了部分与硬件相关的低级汇编代码，AlienOS完全由Rust进行编写，为了进一步验证IKA的通用性，我们将IKA应用到Linux系统中，并将已有的Rust内核模块改造为隔离域。

[Alien OS](#)与 [linux-kernel-module-rust](#)在github上开放并免费提供。



5.1.1 Alien OS Prototype

AlienOS大约包含30k行Rust和200行左右的汇编代码，拥有完整的文件系统和网络堆栈。其中，TCB包含7k行代码(3k行是域的基础设施代码)，完全由safe rust实现的域集合包含23k行代码，占74%。如图所示，IKA的三个核心部分自底向上构成了Alien OS的完整形态。其中，TCB中的runtime负责初始化硬件并加载各个隔离域，除此之外的大部分内核功能全部被划分为独立的域进行实现，包括任务管理，调度器等核心模块。我们划分了几个不同的集合：通用域集合、文件系统域集合、驱动域集合，每个集合中都包含了多个域，它们都依赖Domain lib提供的安全抽象。许多域都可以在运行中实现热升级，比如调度器。从零开发允许我们完全按照IKA的倡议和方法使用Rust实现一个功能完整的操作系统，从而验证基于语言机制实现模块隔离和热升级的可行性。

Type	Code Lines
runtime	3.7k
domain lib	3.3k
domains	23k

5.1.2 Linux Kernel Module Modification

Linux内核模块修改在Linux中使用Rust进行编程依然处在早期尝试阶段，我们的工作建立在Rust For Linux项目【】上。我们使用Rust实现了2k行的内核模块作为IKA的runtime，RFL数据结构的安全抽象被我们迁移到domain lib中, 并根据第4.2节的设计增加了大约1k行代码用于封装内核API和添加热升级所需支持。我们还分别修改了rnull(150行), rnvme(300行)两个使用Rust实现的lkm，以将其改造为隔离域。

与从零开发的AlienOS不同，Linux kernel已经是一个功能完善的复杂系统，因此我们将IKA作为Linux的扩展进行实施。值得一提的是，Linux和AlienOS共享相同的隔离和热升级所需的基础设施代码(图)以及自动化工具。鉴于两个操作系统巨大的差别，在runtime和domain lib中，我们修改了系统相关的实现，比如，runtime中，我们使用现有内核模块提供的功能为域的加载分配虚拟地址空间，在domain lib的安全抽象中，我们不需要像Alien OS一样直接处理硬件设备的不安全操作，因为这些不安全操作在Linux的现有内核模块中已经经过了抽象，因此，我们直接对现有的C接口和数据结构进行封装和安全抽象。尽管当前在linux中TCB的全部代码数量(10k, 包括3.5k的域基础设施代码)是域代码数量(2.7k)的三倍，但随着域数量的增多，更多的代码将会由safe rust构成。

Redirect Callback Function to Domain Interface

重定向回调函数到域接口在Linux中，内核可加载模块的常用做法是通过向内核注入回调函数来扩展内核的功能。模块可以使用内核导出的符号，向内核申请资源或者释放资源。与LKM不同，域不能通过注册回调函数来扩展kernel的功能，因为IKA要求对域功能的访问全部从域接口进入。

为了让所有对域功能的访问通过域接口，我们禁止域注册回调函数。我们通过创建中介函数来把内核回调函数重定向到域接口上来解决这个问题, 这基于对内核回调函数接口的观察结果上：这些回调函数参数指定的数据结构中，通常存在一个字段用于保存实现者的私有数据。具体而言，我们按照内核回调函数接口定义来定义域的接口，以尽可能减小两者的差异，在域初始化内核数据结构期间，域对象会被保存在私有数据字段中，在内核调用回调函数时，首先调用我们定义的中介函数，在中介函数中，我们从私有字段中取出域对象，并调用域的接口，当这些数据结构被销毁时，我们从中释放域对象。

kernel modification 我们还修改了内核的编译配置，我们观察到在故障恢复期间以及驱动初始化设备期间，内核发生了崩溃，其原因是任务内核栈发生了溢出。通常，Linux kernel 为每个任务分配大小为8k的内核栈，这在绝大部分情况下是合理的。然而，Rust编译器尝试在栈上为堆变量分配空间，并在稍后拷贝到堆上，这导致内核栈被快速地消耗掉。我们增大了内核分配给任务的栈大小来解决这个问题。当前Rust社区正在对编译器进行优化以减小Rust程序对栈空间的不必要消耗。

7. Evaluation

在评估时，我们想要回答以下问题：

- 1. 为支持模块隔离和热升级，IKA的域抽象带来的开销有多大？(第7.2节)
- 2. 故障恢复如何影响应用程序以及需要付出什么代价？(第7.3节)
- 3. IKA的热升级如何发挥作用，热升级的服务暂停时间有多大？(第7.4)

7.1 Setup

对于Linux 驱动的隔离域改造，所有基准测试均在一台配备了Intel i9-13900HX处理器、32GB RAM的服务器上运行，服务器运行64位Ubuntu22.04，内核版本为经过简单修改(第5.2节)的linux 6.8.0，该服务器配备了一块1TB Samsung SSD 990 PRO 。在运行测试期间，所有CPU核心运行在performance模式，并且禁用了超线程，TurboBoost以及CPU频率缩放。测试所用的核心被固定在2.1GHz以减少性能测试中的差异。在对NVMe SSD进行测试时，SSD会被提前进行格式化，以便每次测试时它都处于可比较的状态。我们还在AlienOS原型上展示了对任务调度器进行热升级的可行性，对于Alien OS原型，测试在一块搭载4核1.5GHz riscv处理器，8GB内存的的开发板上进行。

7.2 Isolation Overhead

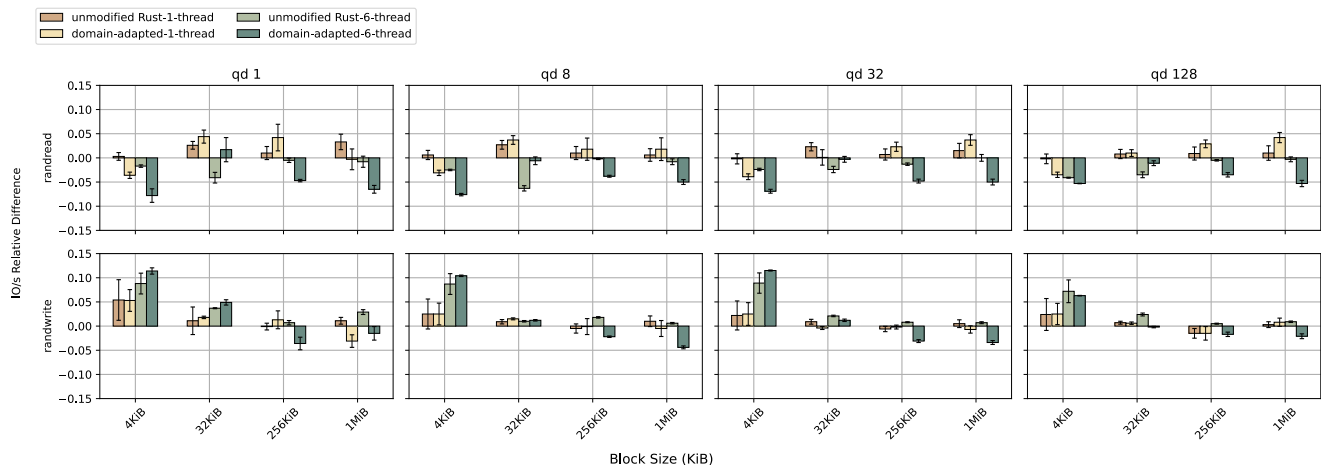
域调用的开销 cross-domain invocation

Operation	Cycles
RedLeaf (no arguments)	113
RedLeaf (passing an RRef<T>)	134
RedLeaf via shadow (no arguments)	290
RedLeaf via shadow (passing an RRef<T>)	326
IKA (no arguments)	50
IKA (passing an DBox<T>)	99
IKA (no arguments, no live update proxy)	5
IKA (passing an DBox<T>, no live update proxy)	49
MPK (single instruction)	40
System call (getpid)	270

为了更详细地了解IKA与同样利用语言机制进行隔离的方案(RedLeaf[])的区别。我们进行了一个微基准测试，以测量在进行域间调用时携带不同数量参数和不同配置下的开销。我们的测试接口只是简单地对传入的参数进行自增并返回结果。表二【】显示了以CPU周期为单位的调用域接口的成本，包括无参调用以及单个共享对象传递调用。其中，RedLeaf允许使用shadow机制以提高隔离模块的可靠性，IKA默认支持隔离模块的热更新，也允许移除热更新功能。为了便于参考，我们还统计了简单系统调用(getpid)开销和使用最新隔离机制MPK的指令执行开销。

测试结果表明，在全功能状态下，与同样利用语言机制进行隔离的RedLeaf相比，IKA可以将域间调用的开销降低到100个周期内。这些改进主要来自两个方面，首先，IKA不需要通过保存任务状态来实现域的故障隔离，而任务状态的保存和恢复会在每次域间调用上带来80-90个周期的开销。此外，IKA不用引入shadow domain来重启故障域，因为IKA可以通过语言级机制处理域故障并通过热更新来修复域错误。我们在实验中还观察到，IKA的域间调用开销中很大部分来自对热升级的支持，在移除热升级代理之后，域间调用下降到50个周期内。

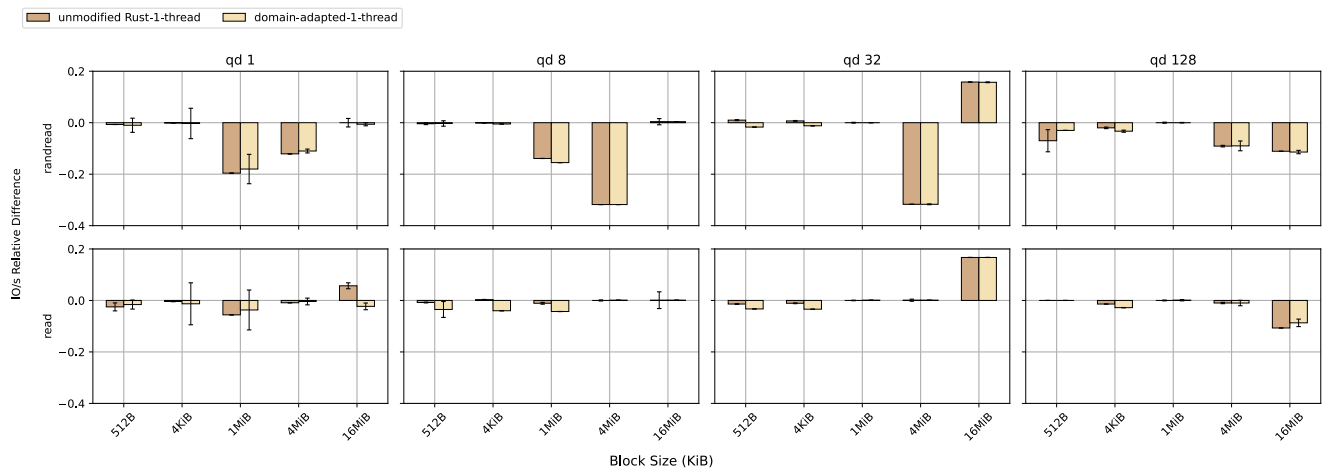
即使是最简单的系统调用(getpid)，也要花费大概270个周期，相较而言，利用语言机制的方法大大减少了上下文切换带来的开销。Intel CPU引入的新的隔离硬件MPK可以用来实现高效的进程内隔离，单个MPK指令花费大约40个周期，而完整的一次切换通常需要99-105个周期【】【ERIM】。与MPK类似的技术PKS具有类似的开销。遗憾的是，它们的保护数量受到页表可用位的限制。IKA实现了现代高效隔离硬件类似的开销，但IKA不受硬件约束。其它的虚拟化隔离方案将会带来更大的开销，比如，基于EPT的隔离一次调用需要396个周期【】。



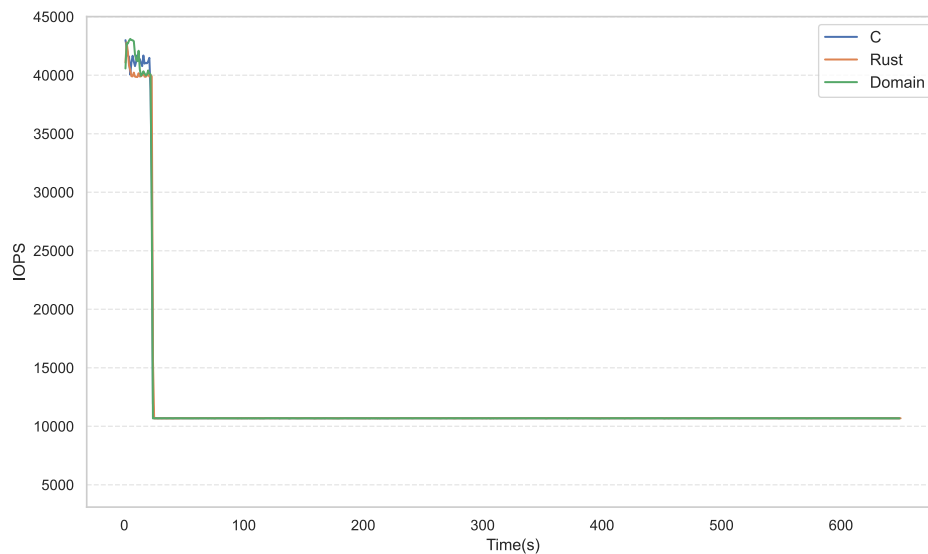
IKA支持低开销的隔离。在基于语言机制实现隔离的方法中，隔离的开销主要由两个部分构成：故障隔离代理生成的故障捕获代码和热升级代理执行的任务统计以及共享堆数据所有权转移。在Linux系统中，还包含了使用接口重定向(第6.2节)带来的转换开销。为了了解Rust的安全性是否适用于开发现代操作系统内核中的高性能驱动程序，我们测量了Linux中的两个设备驱动程序的隔离开销：null block 和 Nvme，每个驱动使用不同的存储介质。null block使用内存模拟磁盘，因此其性能不受设备本身的限制，而nvme驱动则使用非易失性存储介质，其性能易受到设备运行状态的影响。

在两个实验中，我们使用fio来生成读写请求。为了为评估提供一个最佳基线，我们设定了fio的配置参数从而为读写提供最小的延迟，以便更好强调隔离的开销。具体来说，我们使用fio的io_uring引擎来提交多个请求，设置直接I/O标志以绕过页面缓存，并使用轮询模式。我们将改造为域的驱动与未被修改的Rust驱动和C实现的驱动进行比较（下文统一称为域驱动，Rust驱动，C驱动）。我们计算域驱动和Rust驱动相对于C驱动的性能优劣。

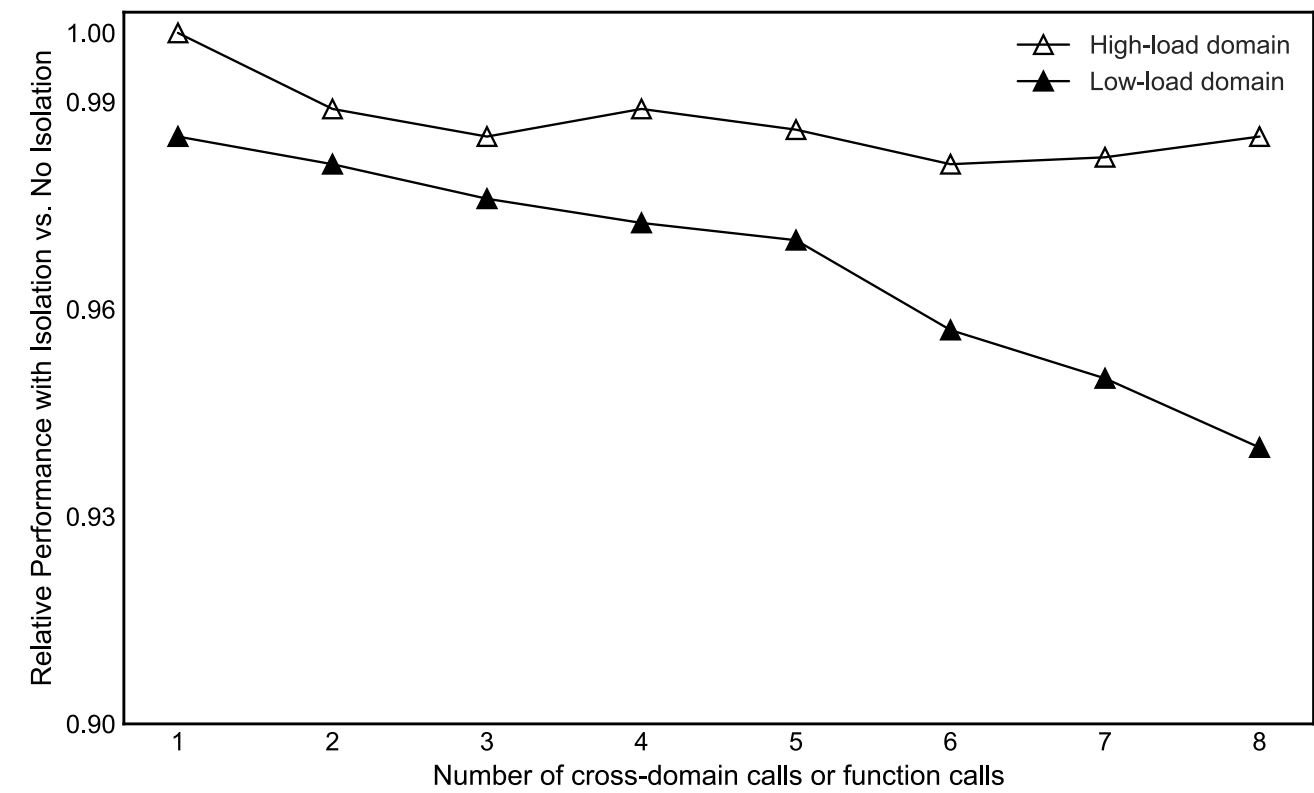
null blk null blk设备使用系统内存作为后备存储介质。由于其需要使用内核内存，在测试前，我们首先通过写磁盘提前分配内存以消除内核内存分配造成的影响。我们在不同的队列深度(qd)、线程数和块大小配置(block size)下执行多次随机读写测试(图?)。就从使用Rust实现设备驱动的角度看，在随机读测试中，Rust驱动的总体性能与C驱动类似，吞吐量最多分别降低4.1%和提高3.3%；在随机写中，Rust驱动的总体性能优于C驱动，我们检查了Rust驱动和C驱动的具体实现，C驱动在处理写请求中实现了更多的功能，比如检查是否使用缓存以及修改页标记等，我们将Rust驱动的优势归功于更短的代码路径，尽管如此，我们仍然可以得出结论，使用Rust实现高性能驱动是可行的。修改后的域驱动与Rust驱动相比，两者的性能在不同的配置下各有优略。在随机读测试中，32个不同配置下有18个配置域驱动的性能发生了下降(1.3%-6.2%)，在随机写测试中，19个配置下域驱动的性能发生了下降(0.1%-5%)，就整体而言，在随机读测试下，域驱动带来了平均1.2%的开销，在随机写测试下则带来了平均0.9%的开销。这说明为支持隔离而额外引入的代码并没有显著降低程序的性能。



Nvme 在nvme设备上的结果如图()所示, 我们没有与null块设备一样在多种配置下测量随机写的性能, 因为nvme设备的写性能会严重受到设备因素的干扰, 鉴于时间有限, 我们还没有在Nvme驱动程序中添加多线程的支持, 因此我们删去了多线程的测试。在大多数配置下, Rust驱动同样获得了与C驱动相近的性能。当块大小增大到1MB-16MB, Rust的性能波动会突然变得很大, 在qd=32,blk_size=4MB的配置下, Rust驱动的性能下降了32%, 在qd=32,blk_size=16MB配置下, Rust驱动的性能反而提升了15%。我们检查了两个驱动的实现, 并没有发现可能与这些变化相关的的实现差异, 我们将对此更进一步的分析留作未来的工作。在引入隔离的情况下, 域驱动与Rust驱动几乎没有差异, 随机读的平均开销可以忽略不计(0.09%), 顺序读的平均开销为0.9%。我们单独测量了随机写的性能差异, 图()显示了完整写完一次磁盘的吞吐量变化, 由于受到运行时间和设备缓存的影响, 三者在前30s内性能都产生了不同程度的波动, 但差距保持在2%以内, 在经过30s左右的高速写入后, 设备性能快速下降并达到稳态, 此时三者的总体性能几乎一样。也就是说, 隔离带来的开销相比硬件设备本身的开销, 可以忽略不计。



Multi Domain

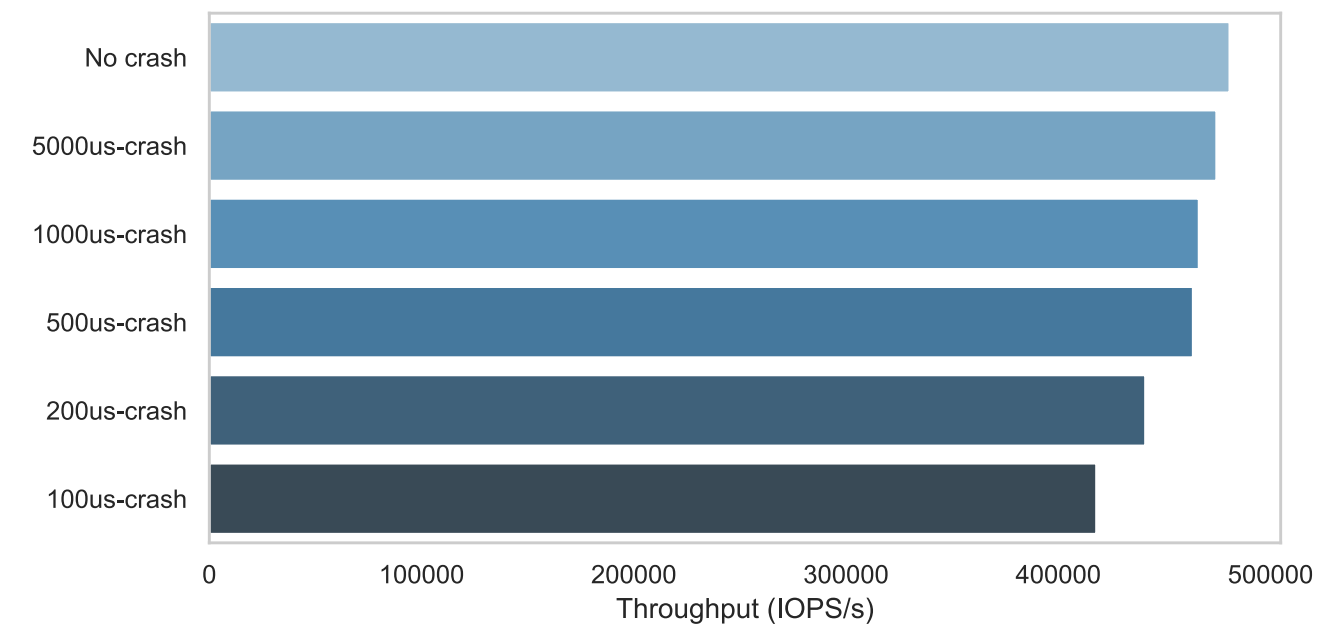


随着较大的内核子系统被分解为多个隔离域，其中一些隔离域的实现就需要依赖其它隔离域的功能。同时，鉴于对安全性要求的提高，原来可能位于一个域中的功能也会被拆分到其它域中。在这两种情况下，域间通信就会变得更加频繁。为了评估任务在一次请求中执行多次域间调用而产生的开销，我们设计了这个实验。首先，我们在块设备驱动域中实现了一个函数，用于对缓冲区进行随机填充，然后，我们额外实现一个虚拟的隔离域，并且在域中实现缓冲区随机填充的功能。在驱动域中，在进行真正的读写前，我们以两种不同的方式调用缓冲区随机填充功能：(1) 调用虚拟域的接口，这相当于在隔离模式进行域间调用；(2) 直接调用函数实现，这等价于传统的没有进行隔离划分的情况。

我们针对512B和4KB块大小的请求调用缓冲区随机填充，然后通过重复调用来模拟不同数量的域间调用或者函数调用，并计算隔离情况下相对于无隔离的性能比例。我们在null blk驱动上统计了吞吐量，实验结果如图()所示。在512B块大小的请求下，随着域调用次数的增加，吞吐量占比逐渐下降，其中，单次域调用导致吞吐量下降1.5%，而执行8次域调用吞吐量下降了6%。开销的来源是显而易见的，首先，512B缓冲区的随机填充所需要的时钟周期很小，小于因为隔离需要而引入的额外代码，其次，直接函数调用给编译器提供了函数内联的机会。4KB块大小的结果也验证了这个结论。当虚拟域执行4KB大小缓冲区的随机填充时，单次域调用和8次域调用的吞吐量分别下降了下降了0.01%和1.5%，远小于512B的情况，因为此时随机填充占据了更多的CPU时间。

对于隔离域来说，512B和4KB的缓冲区随机填充本质上代表了域内执行的不同大小的工作负载。这也反过来给我们进行域划分提供了一些很有价值的反馈，即尽量避免将较小的功能划分到单个域中，而是尽可能让独立的域承担一定的工作量，从而在安全性要求和性能之间取得折衷。

6.3 Error Recovery



故障隔离

IKA的故障隔离对许多需要保证可靠性的服务至关重要【1】。一方面，正确的故障处理有助于确保内核模块不会发生资源泄露或死锁从而停止运行；另一方面，借助故障隔离代理，我们可以对用户隐藏模块崩溃的发生并透明地进行调用重启。但是如果故障隔离的开销过大，以至于和用户程序直接暂停服务导致的结果一样，那故障隔离就失去了意义。

因此，为了评估故障恢复对用户程序性能的影响，我们开发了一个测试，我们在设备驱动域中定时地触发错误来模拟内核模块在运行过程中可能的故障情况，这会触发故障恢复流程。我们调整了错误触发的间隔，以确保重启的调用不会再次触发错误，然后使用FIO作为基准应用程序并按照4KB块大小连续读取块设备。我们设置了不同大小的错误发生间隔，结果如图()所示。故障恢复会使得I/O吞吐量下降。在无故障和在100us故障频率下，读取的吞吐量分别为480k/s和418k/s，性能下降了13%，吞吐量随着故障频率的升高而降低。

我们统计了故障恢复花费的时间为20-25us，因为故障恢复需要执行堆栈展开并释放分配的资源，而这需要解析和扫描大量的DWARF信息。相较而言，在我们的设置中，完成单个I/O请求的时间平均为2us，因此故障频率的升高会导致更多的I/O请求遭受故障恢复的影响。在没有隔离的情况下，故障将会直接导致用户程序崩溃，相比之下，即使在100us这样高频率下，故障恢复带来的开销依然在可以接受的范围。

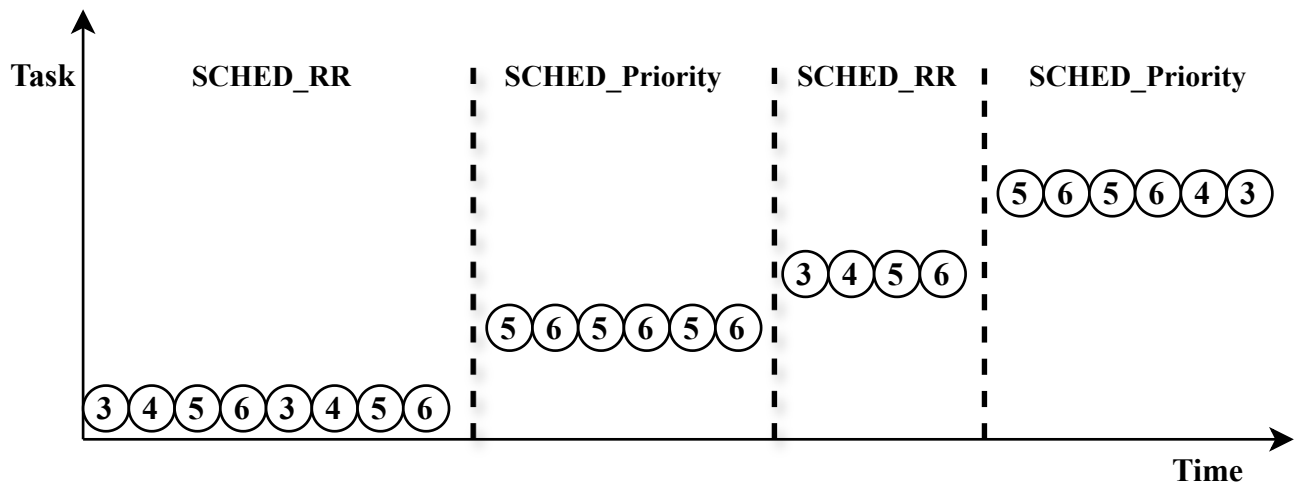
Type	rnull	rnull-LTO	rnvme	rnvme-LTO
Text	68.00 KiB	68.00 KiB	136.00	132.00 KiB
Data	8.00 KiB	4.00 KiB	12.00	12.00 KiB
Bss	4.00 KiB	4.00 KiB	4.00	4.00 KiB
Rodata	12.00 KiB	8.00 KiB	16.00	16.00 KiB
.rela.dyn	6.19 KiB	4.90 KiB	11.88	12.52 KiB
.eh_frame	10.18 KiB	6.75 KiB	20.78	12.57 KiB

Type	rnull	rnull-LTO	rnmve	rnmve-LTO
.eh_frame_hdr	2.13 KiB	1.21 KiB	3.75	2.10 KiB
.gcc_except_table	3.93 KiB	3.54 KiB	8.78	7.56 KiB
Total(unwind)	114.43- 16.24(14.2%)	100.4- 11.5(11.4%)	213.19- 33.31(15.6%)	198.75- 22.23(11.2%)

故障隔离在时间上的开销可以接受，我们统计了内核模块的大小，以确认故障隔离是否会在空间上带来较大的开销。我们统计了两个域驱动程序，并且对需要加载到内存中的不同部分进行了区分。结果如表()所示，其中.eh_frame、.eh_frame_hdr、.gcc_except_table是实现故障隔离所需的部分，.eh_frame_hdr是为了在堆栈展开期间加速函数查找而生成的，即使去掉也不影响。在不开启更高级别优化的情况下，这三部分内容分别占到了整个可执行文件大小的14.2%和15.6%。链接时优化（LTO）是一种整体程序优化技术,可以提高程序的运行速度并减小可执行文件的大小。在LTO优化下，整个可执行文件大小分别减小了12.3%和6.7%，而这三部分的大小也因为优化而减小，分别只占到整个文件大小的11.4%和11.2%。我们认为实现故障隔离所导致的额外空间开销相比其带来的好处是可以接受的。

6.4 Living Update

Example



热升级的表达性

在这个实验中，我们评估隔离模块的热升级能力。由于我们暂时无法使用Rust实现Linux的核心模块，因此，我们使用案例研究（如 Baumann 等人【】）在AlienOS中进行实验。我们以调度器模块的动态替换为例，验证IKA在实施核心模块热升级方面的可行性与有效性。之所以选择这个模块，是因为现代操作系统的调度器通常无法满足用户特定的工作负载【】【】，而由于与内核的结构和其它模块高度耦合，对调度器的升级的大多数情况下都需要重启系统。

我们的实验设计了包含多个用户任务的场景，这些任务执行矩阵计算，其中任务5和6被赋予最高优先级（Priority=16），其余任务为普通优先级（Priority=0），初始调度策略为实时轮转调度（RR），任务按轮转顺序平等获取CPU时间。通过触发四次热升级操作（RR↔Priority循环切换），观测系统在升级过程中的行为：当切换至优先级调度策略时，高优先级任务5和6立即抢占CPU资源，并且会一直占有CPU，其余任务进入等待队列；而回退至RR策略后，所有任务恢复公平轮转，且历史运行进度（如任务的矩阵计算迭代计数）在升级前后完全一致，未出现状态丢失或计算重复。在整个测试运行过程中，我们没有重启系统。实验结果表明，IKA能够处理组件级别更新。

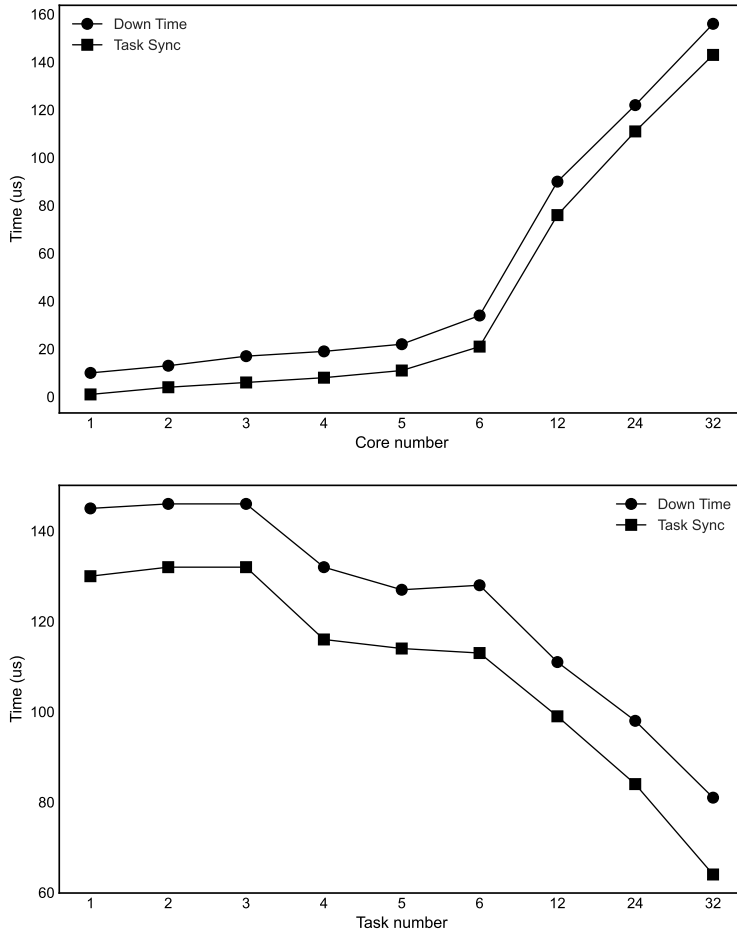
Downtime

服务暂停时间

热升级的真正作用在于减少系统服务暂停的时间，我们统计了在Linux系统上更新一个有状态域和无状态域的时间分布。与现有的实施热升级的作品不同，在IKA中，隔离模块的热升级建立在基于语言机制的隔离上，并使用特殊的状态保存和同步机制，这使得该方法当前只能针对Rust开发的模块，因此，很难将IKA的热升级能力与其它方法进行直接对比，因此我们独立分析IKA的方法并与其它方法进行理论上的比较。

1	Phase	Block Device Driver(State)	
	Logger(No state)		
2	Total Time	1070 us	760
	us		
3	Load new Domain	324 us	300
	us		
4	Down Time	146 us	x
5	Task Sync	134 us	x
6	Doamin reinit and State transfer	12 us	122
	ns		
7	Domain replace	347 ns	200
	ns		
8	RCU Sync	x	27
	us		
9	Resource cleanup	576 us	410
	us		
10	Other	24 us	23
	us		

作为示例，表（）显示了使用IKA进行热升级的时间分解。我们使用块设备驱动域和一个无状态日志输出域作为典型的测试用例。我们我们将热升级划分为多个阶段，有状态和无状态略有区别，其中有状态的热升级包含Task 同步阶段而不包含RCU同步，无状态的热升级相反之， Task sync 、 Doamin reinit and State Transfer 和 Domain replace 构成了升级过程中的服务暂停时间。正如所观察到的， IKA能够在1ms左右的时间更新一个有状态域。值得注意的是，在热升级的大多数阶段，例如新域加载，资源清理，正在运行的任务不会受到影响。在服务暂停阶段，只有涉及到被更新的域的任务会暂停执行，服务暂停时间仅仅占整个过程的14%(146us)。我们将此归功于IKA中数据状态的分离存储和高效的任务统计和检查方法。对于无状态域，整个升级期间都不会中断当前正在运行的服务，因此不存在服务暂停时间，其花费的大多数时间是用于新域加载和资源清理。根据【Plugshed】的报告，使用【kpatch】升级一个内核模块导致的服务暂停时间在2800-3000us左右， 这比IKA高出了20倍，原因在于其需要暂停系统中的全部CPU并且扫描系统中的所有任务的堆栈，并且kpatch还不支持组件级别的热升级。



在IKA中，热升级的时间跟系统中的CPU密切相关，因为在统计位于域中的任务数量之前，需要向其它CPU发送一次IPI。在本实验中，我们以不同核心数的块设备驱动热升级为例，展示IKA的热升级和CPU数量的关系。如图所示，我们通过CPU热插拔关闭系统可用的CPU，当系统中可用CPU数量从1增长到32时，用于任务同步的时间从1us增长到143us，服务暂停时间从10us增长到156us，整个过程服务暂停时间中用于 `Doamin reinit and State transfer` 和 `Domain replace` 的部分几乎保持在9-14us。对于任务同步来说，CPU数量的增加意味着IPI数量的增加。实验中，我们观察到任务同步所增长的时间与CPU增长的数量并不一定是线性相关的，产生这个的原因是一个微妙细节：如果CPU此时位于用户态，那它因为IPI中断所花费的时间(4-5us)就比当前处于内核态中的CPU(2-4us)多，因为用户态的CPU在处理中断的过程中需要进行特权级切换并保存更多上下文，而位于内核态的CPU也会根据是否处于idle状态而花费不同的时间。

图()右显示了当前访问域的任务数量增加时的任务同步时间和服务暂停时间。在任务数量从1增长到32的过程中，任务同步花费的时间从130us下降到了64us。因为随着任务数量的增加，更多CPU将会有更大的概率处于内核态中，从而降低IPI花费的时间，进而降低任务同步的时间。不过，如图展示的是一种比较理想的状态，在大多数情况下，服务暂停时间都保持在100-200us之间。

7. Related Works

基于硬件的隔离 基于硬件的隔离方案【】【】【】【】通过硬件特性直接在物理层面实现对内核模块的隔离。例如，Intel VT-x【】和AMD-V【】等硬件虚拟化技术允许虚拟机管理器（Hypervisor）高效地运行多个虚拟机，并在硬件层面隔离彼此的资源。然而，这些技术对实施普通内核模块的隔离过于昂贵，随着隔离模块数量增多，这些方法也无法进行扩展。最新的硬件架构【】【】【】探索了以内存标签和功能为中心的细粒度隔离的想法，比如Intel MPK通过使用4位保护密钥（保存在页表条目的未使用位中）标记各个页面，在单个地址空间内强制实施隔离。受限于MPK只能作用在用户态内存页上，【】【】【】等使用MPK在内核中进行隔离的方案针对的是unikernel形态的单特权级操作系统。IKA基于语言机制实施隔离，不需要依赖特定的硬件，隔离模块的数量不受限制，因此可以对内核模块进行细粒度的隔离。

基于软件的隔离 基于软件的隔离【】【】【】通过逻辑层面的机制在操作系统中实现模块分离和限制。其中，SFI是一种经典的技术，通过插入边界检查指令对内存访问和代码执行行为进行约束，以防止非可信代码访问未授权的内存区域或执行恶意操作。SFI在支持多种编程语言隔离的能力、降低隔离的开销和跨模块通信的成本之间取得了平衡【】。不幸的是，如果没有适当的硬件支持，SFI的开销仍然很高。一方面，SFI依赖编译器插件生成检查代码，由于编译器无法感知程序的功能，只能机械地为所有可能的代码生成检查，这会导致程序性能下降，另一方面，SFI依赖开发者的手工工作以指定SFI作用的模块和功能。IKA使用Rust语言的类型和内存安全对内存访问和代码执行行为进行约束，这只会为必须的代码生成边界检查。IKA提供实现隔离所需的基础设施以及自动化工具，开发者不需要关注隔离的内部细节。

微内核架构 微内核架构【seL4】【MINIX3】【】通过将操作系统的核心功能极度精简，仅保留进程管理、内存管理和进程间通信（IPC）等基本功能，其余模块被移至用户态作为一个服务运行。这样的设计实现了明确的模块边界，每个模块都通过受控的IPC进行通信并实施进程的地址空间隔离，这还在一定程度上简化了实施组件级更新的难度。然而，由于服务之间的频繁通信需要多次上下文切换，通信开销较高，微内核也与当前的宏内核无法兼容。IKA在同一个地址空间内实施模块隔离，因此模块间通信不需要上下文切换，也不需要改变内核形态。

热升级 热升级是一种在重启系统的情况下对内核模块进行更新的技术，分为功能级别【】【】【】和组件级别【】【】两种主要方案。功能级别的热升级允许开发者通过补丁形式直接修改或替换内核中的特定函数，完成漏洞修复或功能增强，而无需重新启动系统。功能级别的优点在于其实现相对简单，只需将变更限定在特定的功能模块内。相比之下，组件级别的热升级旨在对整个模块进行全面替换，通过保存旧版本的运行状态，并在新模块加载后迁移状态来实现无缝切换。实现内核模块的热升级的核心在于维护升级前后的状态数据的一致性。一些作品使用对象替换、阴影数据结构和状态重建等技术来实现状态数据迁移【】【】【】，然而，不同模块内部的状态数据可能包含复杂的数据结构，这些作品提供的解决方案不够通用。与这些方法不同，IKA为所有模块提供了状态保存和恢复的底层数据结构和接口，并负责自动迁移状态数据，从而允许不同模块轻松地实现热升级。

8. Conclusion

我们介绍了IKA，一种使用语言机制实现内核模块隔离和热升级的方法，用于提高内核的安全性、可靠性和可扩展性。在Rust语言提供的安全保证基础上，基于系统分解和模块化思想，IKA提供了一个通用的用于实施模块隔离的框架，IKA利用私有堆和共享堆的设计来实现模块间的内存隔离和通信，IKA还通过故障隔离加强了内存隔离的有效性。在模块隔离的基础上，IKA提供了对隔离模块的组件级热升级支持，这依赖IKA提出的状态分离存储和高效的状态同步方法。我们通过实现一个原型操作系统Alien OS来验证我们的方法，更重要的是，我们将这个方法应用在现代商用操作系统Linux上。实验表明，IKA是一个通用的架构，其隔离具有很小的开销，Linux隔离驱动的I/O吞吐量平均仅下降1%-2%，IKA的组件级热升级导致的服务暂停时间很短，在100-200us左右。未来，在已经实施的基础设施上，我们希望可以为更多的Linux内核模块实施基于语言机制的隔离和热升级。